

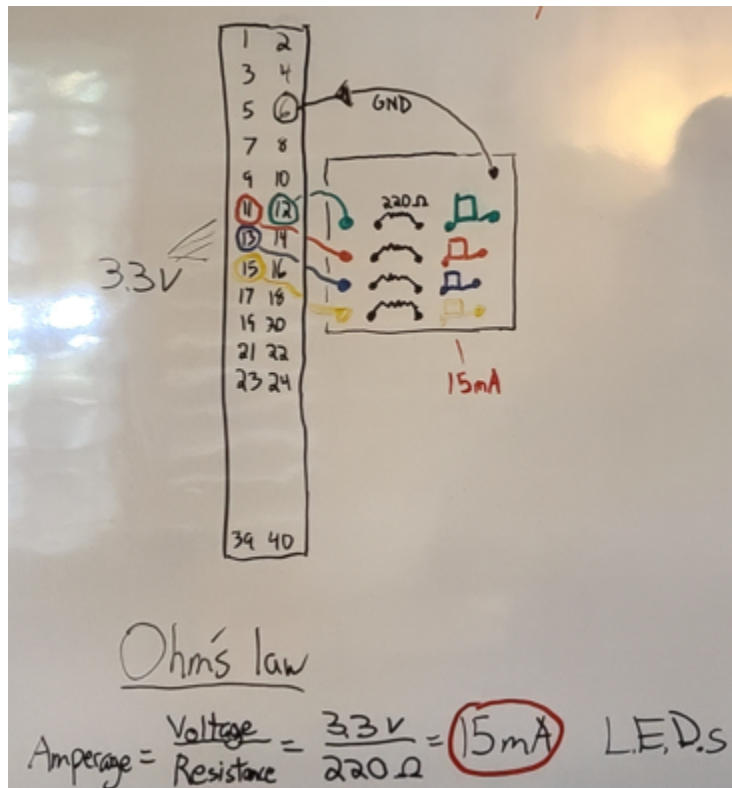
Final Project

3. Randoms - Devise your own random number generator (don't use the rand() function from C/C++ and do not use Linear Congruence algorithm either - it too simple to implement). Demonstrate your random number generator on the breadboard by lighting up LEDs as you see in an intuitive manner.

Presentation video:

<https://www.youtube.com/shorts/IOt3fKANgtU>

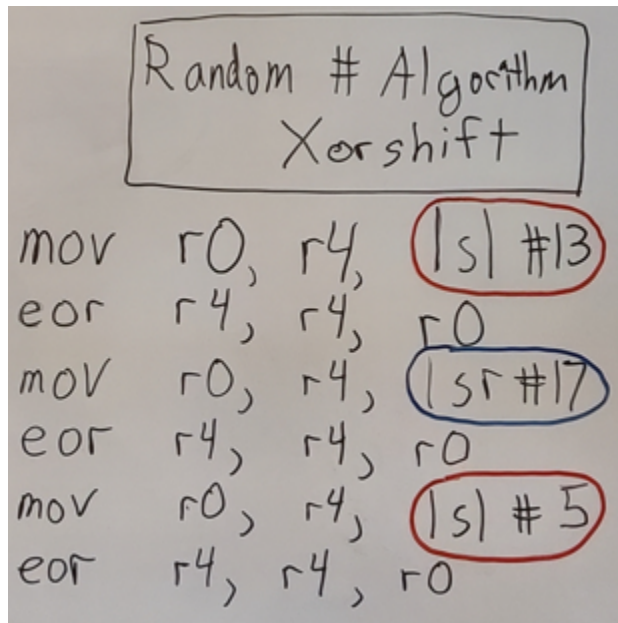
Circuit setup:



I am using GPIO pins 11,12,13,15 (3.3v power) with each pin linked with a 220ohm resistor and an LED. All 4 LEDs are then linked to pin 6 for the Ground.

According to Ohm's law, each LED has 15mA of current.

Xorshift algorithm:



Handwritten assembly code for the Xorshift algorithm, enclosed in a box titled "Random # Algorithm Xorshift". The code consists of six lines:

```
mov r0, r4, (lsl #13)
eor r4, r4, r0
mov r0, r4, (lsl #17)
eor r4, r4, r0
mov r0, r4, (lsl #5)
eor r4, r4, r0
```

The shift amounts 13, 17, and 5 are circled in red in the original image. The register r0 is used for intermediate results, and r4 is used for the main value being shifted and XORed.

I am using the low 4 bits in r0 to tell my LEDs whether they should be on (1) or off (0).

I am using r4 to store the rng value.

Line 1: take value in r4 and shift left by 13 and then store in r0

Line 2: Xor (combine) the values in r4 and r0 and store them in r4

Line 3: take value in r4 and shift right by 17 and then store in r0

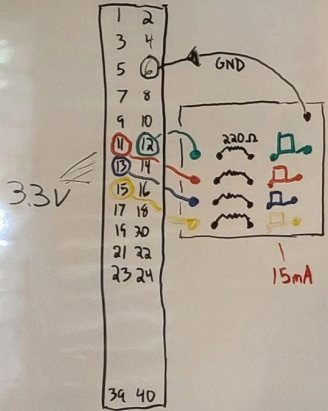
Line 4: same as line 2

Line 5: take value in r4 and shift left by 5 and then store in r0

Line 6: same as line 2

Designing my circuit

Michael Donnelly



3.3V

220Ω

15mA

Random # Algorithm
Xorshift

```
mov r0, r4, (1s) #13
eor r4, r4, r0
mov r0, r4, (1sr) #17
eor r4, r4, r0
mov r0, r4, (1s) #5
eor r4, r4, r0
```

Ohm's law

$$\text{Amperage} = \frac{\text{Voltage}}{\text{Resistance}} = \frac{3.3\text{V}}{220\Omega} = 15\text{mA} \text{ LEDs}$$

Testing my circuit by using Terminal with commands:

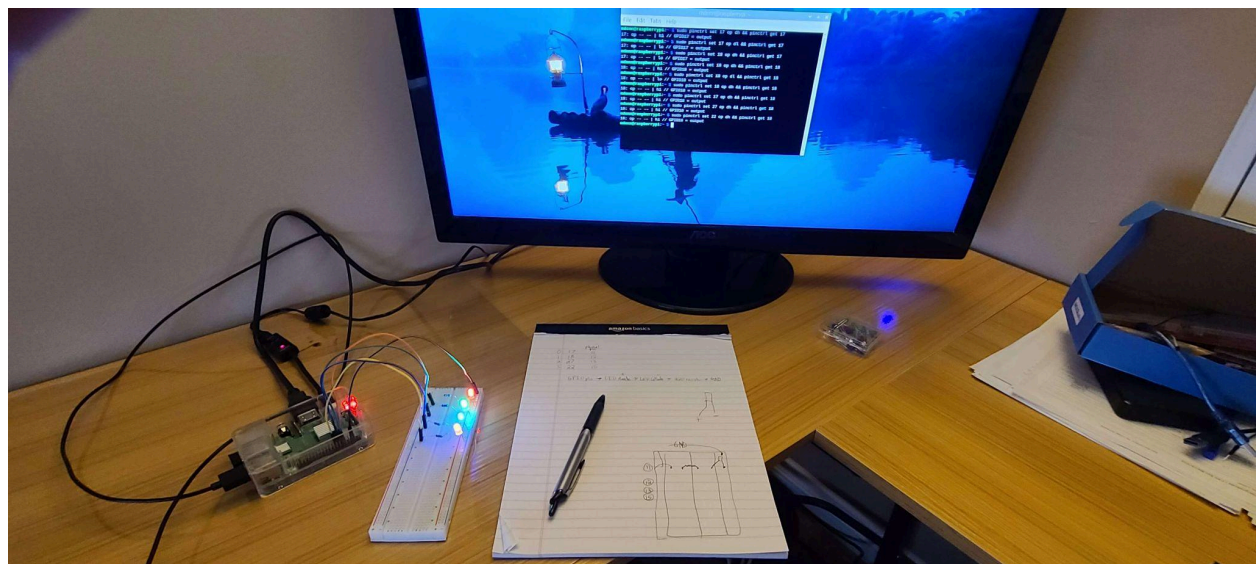
```
Sudo pinctrl set 17 op dh && pinctrl get 17
```

```
Sudo pinctrl set 18 op dh && pinctrl get 18
```

```
Sudo pinctrl set 22 op dh && pinctrl get 22
```

```
Sudo pinctrl set 27 op dh && pinctrl get 27
```

Which turns on physical pin #11,12,13,15 and then queries status of the pin (hi / lo).



Things I learned while making this project:

This project took a lot of work. I made many mistakes with the circuit board. I accidentally plugged my GND into pin #2 (5v power). I also didn't realize that the LED legs need to be in different rows. I had to use a Python program to test the LEDs because I initially couldn't get ARM code to run.

I then found out that I had x64 Raspian installed so I reinstalled the x32 version. However, now the pi was not recognizing pinctrl and digitalwrite calls so I installed the pinctrl github repo and it was as simple as typing a single line in the Terminal.

I learned how to use SSH to send the file from my desktop directly over to the Pi. So whenever I had to tweak the file, I could just send my latest desktop version.

I didn't know how to use a delay command in the ARM code so I used a manual delay by forcing it to run useless cycles.

Then I learned how to run the program from the Terminal which also means I could technically have ran it headless from my desktop via SSH.